

DEPARTMENT OF ECE, SVIT
DIGITAL SYSTEM DESIGN USING VERILOG

BEC302
MODULE 3
FLIP-FLOPS AND ITS APPLICATIONS

Syllabus: The Master-Slave Flip-flops (Pulse-Triggered flip-flops): SR flip-flops, JK flip flops, Characteristic equations, Registers, Binary Ripple Counters, Synchronous Binary Counters, Counters based on Shift Registers, Design of Synchronous mod-n Counter using clocked T, JK, D and SR flip-flops.

Prepared by: Pavithra G S, Asst. Professor
pavithra.gs@saividya.ac.in

vtu-easy.com

TABLE OF CONTENTS

1. Sequential Circuits – Introduction & Basics

- [1.1 Combinational vs Sequential Circuits](#)
- [1.2 Clock, Synchronous & Asynchronous Networks](#)
- [1.3 Basic Bistable Element](#)
- [1.4 Flip-Flops – Definition & Types of Inputs](#)
- [1.5 Latches](#)

2. SR Latch & $\bar{S}\bar{R}$ Latch

- [2.1 SR Latch \(NOR-based\)](#)
- [2.2 \$\bar{S}\bar{R}\$ Latch \(NAND-based\)](#)
- [2.3 Gated SR Latch](#)
- [2.4 Gated D Latch \(Data Latch\)](#)

3. Master-Slave Flip-Flops (Pulse-Triggered)

- [3.1 Introduction & Need](#)
- [3.2 Master-Slave SR Flip-Flop](#)
- [3.3 Master-Slave JK Flip-Flop \(MS-JK\)](#)
- [3.4 0s Catching & 1s Catching](#)
- [3.5 Master-Slave D Flip-Flop](#)
- [3.6 Master-Slave T Flip-Flop](#)

4. Edge-Triggered Flip-Flops

- [4.1 Positive Edge-Triggered D Flip-Flop](#)
- [4.2 Negative Edge-Triggered Flip-Flops](#)
- [4.3 Asynchronous Inputs: PRESET & CLEAR](#)
- [4.4 Positive Edge-Triggered JK Flip-Flop](#)
- [4.5 Positive Edge-Triggered T Flip-Flop](#)
- [4.6 Master-Slave Flip-Flop with Data Lockout](#)

5. Characteristic Equations of Flip-Flops

- [5.1 SR Flip-Flop Characteristic Equation](#)
- [5.2 JK Flip-Flop Characteristic Equation](#)
- [5.3 D Flip-Flop Characteristic Equation](#)
- [5.4 T Flip-Flop Characteristic Equation](#)
- [5.5 Excitation Tables of All Flip-Flops](#)

6. Registers

- [6.1 Introduction to Registers & Shift Registers](#)
- [6.2 4-Bit SISO Shift Register](#)
- [6.3 4-Bit SIPO Shift Register](#)
- [6.4 4-Bit Parallel-In Shift Register \(PISO/PIPO\)](#)
- [6.5 Bidirectional / Universal Shift Register](#)

7. Counters

- [7.1 Introduction & Terminology](#)
- [7.2 4-Bit Binary Ripple Counter \(Asynchronous\)](#)
- [7.3 Synchronous Binary Counter](#)
- [7.4 4-Bit Synchronous Counter with Parallel Load](#)

8. Counters Based on Shift Registers

[8.1 Ring Counter](#)

[8.2 Johnson Counter \(Twisted Ring / Switch-Tail Counter\)](#)

9. Design of Synchronous mod-n Counters

[9.1 General Design Procedure](#)

[9.2 Design using T Flip-Flop](#)

[9.3 Design using JK Flip-Flop](#)

[9.4 Design using D Flip-Flop](#)

[9.5 Design using SR Flip-Flop](#)

10. Quick Revision – Exam Ready Summary

[10.1 Key Definitions](#)

[10.2 Characteristic Equations Quick Reference](#)

[10.3 Excitation Tables](#)

[10.4 Flip-Flop Comparison Table](#)

[10.5 Counter Design Steps & VTU Exam Patterns](#)

vtu-easy.com

1. Sequential Circuits – Introduction & Basics

1.1 Combinational vs Sequential Circuits

Digital logic can be classified into two major categories: Combinational Logic and Sequential Logic. Understanding the distinction is fundamental to the entire study of flip-flops and counters.

Combinational Logic deals with the technique of combining logic gates into circuits that perform some desired function. The output at any instant of time depends **ONLY** on the inputs present at that instant of time. There is no dependency on past inputs or any form of memory.

Examples of combinational circuits: Adders, Subtractors, Encoders, Decoders, Multiplexers, Comparators, Multipliers.

Sequential Circuits are circuits in which the output depends not only on the present input, but also on the **PAST HISTORY** of these inputs. This means a sequential circuit must possess memory — the ability to store past information.

Examples of sequential circuits: Flip-flops, Registers, Counters, Shift registers, Finite state machines.

Parameter	Combinational Circuit	Sequential Circuit
Definition	Output = $f(\text{current inputs only})$	Output = $f(\text{current inputs} + \text{past state})$
Memory	Not required	Required (flip-flops/latches)
Feedback	No feedback path	Feedback from output to input
Speed	Faster (only gate propagation delay)	Slower (clock-limited)
Complexity	Simpler (no feedback)	More complex (feedback loops)
Building Blocks	Logic gates only	Logic gates + flip-flops
Operation	Arithmetic & Boolean operations	Storing and sequencing data
Example	Parallel adder, MUX, decoder	Serial adder, counter, register

A Combinational network is also described as a two-valued network in which the outputs at any instant are dependent **ONLY** upon the inputs at that instant. It contains no closed loops or feedback paths — hence it is acyclic. The output can be described by a single Boolean algebraic expression.

A Sequential network is defined as a two-valued network in which outputs at any instant depend on the past history or past sequence of inputs. The past history is preserved via feedback from output to input through memory elements. Propagation time delay is the time required for a signal to travel from input to output of a circuit.

1.2 Clock, Synchronous & Asynchronous Sequential Networks

The heart of every digital system is the system clock. The system clock provides pulses without which the sequential system would cease to function correctly. The clock is a periodic waveform — usually a square wave.

Clock Cycle Time: The basic timing interval defined as one period of the clock waveform. It defines the timing interval during which all logic operations must be completed.

Symmetrical Square Wave: High and low durations are equal (50% duty cycle).

Asymmetrical Square Wave: High and low durations are not equal.

There are two basic types of sequential networks:

a. Synchronous Sequential Network: In such systems, a change of state occurs in synchronism with the system clock. A change of state will either occur at the positive transition (rising edge, PT) or at the negative transition (falling edge, NT) of the clock. A circuit that changes state during PT is called a positive edge triggered circuit; that for NT is called negative edge triggered.

b. Asynchronous Sequential Network: There is no synchronization control — a change in input itself can cause a change in output and state immediately. These networks are capable of high-speed performance since it is no longer necessary to wait for the clock. However, memory elements in asynchronous systems are unclocked flip-flops or time delay elements.

Parameter	Synchronous Sequential	Asynchronous Sequential
Memory Elements	Clocked flip-flops	Unclocked FFs or time delays
State Change	Only on clock edge activation	Any input change triggers output
Speed	Slower (clock-limited)	Faster (no clock waiting)
Design Ease	Easier to design and verify	Difficult to design

1.3 Basic Bistable Element

Central to all flip-flops is the basic bistable element. It consists of two cross-coupled NOT gates: the output of the first NOT gate serves as input to the second, and the output of the second serves as input to the first. This creates a feedback loop.

Analysis of the two stable states:

- State 1: Let $x=0 \rightarrow Q = \bar{x} = 1$. Then $y=1 \rightarrow \bar{Q} = \bar{y} = 0$. Check: $\bar{Q} = x = \bar{y} = 0$ and $Q = \bar{x} = y = 1$. Circuit is stable. (1-state or SET state)
- State 2: Let $x=1 \rightarrow Q = \bar{x} = 0$. Then $y=0 \rightarrow \bar{Q} = \bar{y} = 1$. Check: $\bar{Q} = x = \bar{y} = 1$ and $Q = \bar{x} = y = 0$. Circuit is stable. (0-state or RESET state)

Key properties of the basic bistable element:

- It has exactly TWO stable states — hence the name "bistable."
- Since it has two stable states, it is used to store binary symbols. When $Q=1$ the element is said to store a 1 (SET state); when $Q=0$, it stores a 0 (RESET state).
- Q is called the NORMAL OUTPUT; $\bar{Q}$ is called the COMPLEMENTARY OUTPUT.
- A THIRD equilibrium state exists when both outputs are halfway between logic-0 and logic-1 — this is the METASTABLE STATE. It is not a valid logic level and is undesirable.
- When power is applied, the element settles into one of its two stable states and remains there until the input causes a change.

1.4 Flip-Flops – Definition & Types of Inputs

A flip-flop is a bistable device with inputs that remains in a given state as long as power is applied and until input signals cause a change in its output state.

- The process of storing a 1 into a flip-flop is called SETTING or PRESETTING the flip-flop.

- The process of storing a 0 into a flip-flop is called **RESETTING** or **CLEARING** the flip-flop.

Inputs to a flip-flop are of two types:

Asynchronous Input: A signal change of sufficient magnitude and duration that immediately (without waiting for a clock edge) produces a change in the state of the flip-flop. Examples: PRESET (PR) and CLEAR (CLR) inputs.

Synchronous Input: Does not immediately affect the flip-flop state. It affects the state only when a control signal (enable or clock) also occurs. Examples: S, R, J, K, D, T inputs.

1.5 Latches

Latches and flip-flops are both bistable devices and form the basic building blocks of most sequential circuits. The key difference lies in HOW their state changes:

- A FLIP-FLOP changes state only at times determined by the clock signal (on specific edges).
- A LATCH changes state corresponding to changes in input regardless of a clock edge. It is level-sensitive — when enable signal is active (high), output follows input changes continuously.
- When enable is NOT active, a GATED LATCH ignores its inputs and retains the previous state.

Remember: Flip-Flop = Edge-sensitive (changes on clock edge). Latch = Level-sensitive (changes when enable is HIGH). This is the most fundamental difference!

vtu-easy.com

2. SR Latch & $\bar{S}\bar{R}$ Latch

2.1 SR Latch (NOR-based)

The SR latch is the simplest type of latch. It is constructed using two cross-coupled NOR gates. It has two inputs — S (Set) and R (Reset) — and two outputs Q and $\bar{Q}$.

Logic diagram description: Output of upper NOR gate = Q. Output of lower NOR gate = $\bar{Q}$. S is input to lower NOR gate; R is input to upper NOR gate. Each NOR gate output feeds back to the other NOR gate as a second input.

Function Table of SR Latch:

S	R	Q^+	$\bar{Q}^+$	Condition
0	0	Q	$\bar{Q}$	No Change (Memory)
0	1	0	1	RESET State
1	0	1	0	SET State
1	1	0*	0*	Forbidden/Unpredictable

Detailed analysis of all four cases:

Case 1 – S=0, R=0 (No Change): The circuit behaves like the basic bistable element — cross-coupling of two NOT gates. Both NOR gates see their non-feedback input as 0, so they retain their current state. $Q^+ = Q$ (next state = present state). This is the MEMORY state.

Case 2 – S=0, R=1 (RESET): Regardless of $\bar{Q}$ fed to the upper NOR gate, R=1 forces the upper NOR gate output Q to be 0. This Q=0 is fed to the lower NOR gate. With S=0 and Q=0, the lower NOR gate output $\bar{Q} = 1$. Result: Q=0, $\bar{Q}$ =1 — RESET state.

Case 3 – S=1, R=0 (SET): S=1 forces the lower NOR gate output $\bar{Q}$ to be 0. This $\bar{Q}$ =0 fed to upper NOR gate with R=0, gives Q=1. Result: Q=1, $\bar{Q}$ =0 — SET state.

Case 4 – S=1, R=1 (Forbidden): Both NOR gate outputs are forced to 0 (Q=0, $\bar{Q}$ =0 simultaneously). This violates $Q=\bar{Q}$ complementary relationship. When both S and R return to 0 simultaneously, the circuit may enter the METASTABLE state — an unpredictable and highly undesirable condition. This input combination must be AVOIDED.

FORBIDDEN STATE Rule for SR Latch: S=1 and R=1 simultaneously must NEVER be applied to an SR latch. The outputs become 0,0 (which is invalid) and the next state is unpredictable if both inputs go low simultaneously.

2.2 $\bar{S}\bar{R}$ Latch (NAND-based)

The $\bar{S}\bar{R}$ latch (also written as SR-bar latch) is constructed by cross-coupling two NAND gates. It has two inputs $\bar{S}$ (complement of Set) and $\bar{R}$ (complement of Reset), and two outputs Q and $\bar{Q}$.

Note: In an $\bar{S}\bar{R}$ latch, the inputs are active-LOW — meaning a logic-0 activates the function.

Function Table of $\bar{S}\bar{R}$ Latch:

$\bar{S}$	$\bar{R}$	Q^+	$\bar{Q}^+$	Condition
0	0	1*	1*	Forbidden State
0	1	1	0	SET State (active-low $\bar{S}=0$)
1	0	0	1	RESET State (active-low $\bar{R}=0$)
1	1	Q	$\bar{Q}$	No Change (Memory)

Analysis of all four cases:

Case 1 – $\bar{S}=1, \bar{R}=1$ (No Change/Memory): Both NAND gate non-feedback inputs are HIGH, so the outputs are determined solely by the feedback — circuit remains in its current state. This is the HOLD state.

Case 2 – $\bar{S}=0, \bar{R}=1$ (SET): $\bar{S}=0$ forces upper NAND gate output $Q=1$ regardless of the other input. This $Q=1$ is fed to lower NAND gate with $\bar{R}=1$, so $\bar{Q}=0$. Result: $Q=1, \bar{Q}=0$ — SET state.

Case 3 – $\bar{S}=1, \bar{R}=0$ (RESET): $\bar{R}=0$ forces lower NAND gate output $\bar{Q}=1$. This $\bar{Q}=1$ fed to upper NAND gate with $\bar{S}=1$, gives $Q=0$. Result: $Q=0, \bar{Q}=1$ — RESET state.

Case 4 – $\bar{S}=0, \bar{R}=0$ (Forbidden): Both NAND gate outputs are forced to 1 ($Q=\bar{Q}=1$ simultaneously). If both $\bar{S}$ and $\bar{R}$ return to 1 simultaneously, the latch can enter the metastable state. AVOID this condition.

Application: Switch Debouncer using SR latch. When a mechanical switch bounces, the SR latch holds the last stable output state during the bounce period, effectively eliminating the glitches from the Q output. The latch is SET on first contact and holds through any bouncing transitions.

2.3 Gated SR Latch (SR Latch with Enable)

The Gated SR Latch (also called SR latch with enable) overcomes the problem of the latch responding to ANY input change. It adds a CONTROL input C (also called Enable, Gate, or Clock) to the $\bar{S}\bar{R}$ latch using two additional NAND gates.

Operation:

When $C=0$ (Disabled): The outputs of the two additional NAND gates (call them X and Y) are both 1, regardless of S and R. This means $\bar{S}=\bar{R}=1$ for the inner $\bar{S}\bar{R}$ latch — the HOLD condition. The latch is disabled and S, R inputs have NO effect.

When $C=1$ (Enabled): The NAND gates pass S and R to the inner latch with inversion ($X=\bar{S}, Y=\bar{R}$). The latch now responds to S and R inputs normally. Output changes in response to S and R.

Function Table of Gated SR Latch:

S	R	C	Q^+	$\bar{Q}^+$	Condition
X	X	0	Q	$\bar{Q}$	No Change (disabled)
0	0	1	Q	$\bar{Q}$	No Change (hold)
0	1	1	0	1	RESET
1	0	1	1	0	SET
1	1	1	?	?	Forbidden

Since S and R inputs are effective only when C=1 (enable), they are classified as SYNCHRONOUS inputs. Applying S=1, R=1 simultaneously when C=1 must still be avoided.

Timing Diagram Key: When C goes HIGH, latch is TRANSPARENT — output follows S,R changes. When C is LOW, latch retains its state regardless of S and R changes. The dotted line on a timing diagram shows "don't care" instants.

2.4 Gated D Latch (Data Latch / Transparent Latch)

The Gated D latch was designed to overcome the forbidden state (S=R=1) problem of the SR and $\bar{S}\bar{R}$ latches. The solution is simple: place a NOT gate between the S and R inputs so that R is always the complement of S. This single input is called D (Data).

Because there is only ONE data input D, and $R=\bar{D}$, it is impossible for S and R to be simultaneously 1. This eliminates the forbidden state entirely.

Function Table of Gated D Latch:

D	C	Q*	Condition
X	0	Q	No change (latch disabled)
0	1	0	RESET (Q follows D=0)
1	1	1	SET (Q follows D=1)

Detailed operation:

When C=1 (Enabled/Transparent): The latch is transparent — output Q follows input D immediately. If $D=0 \rightarrow Q=0$; if $D=1 \rightarrow Q=1$. The latch is called transparent because the data appears at Q without any delay.

When C=0 (Disabled): The latch is disabled. Q retains whatever value it had when C last went low. Changes in D have no effect.

D Flip-Flop (Transparent Latch): A flip-flop whose output follows its data input D when the clock is active. When clock=1, $Q=D$ continuously. The D latch is sometimes called the D flip-flop in this level-sensitive configuration.

Timing Diagram description: When C is high, Q tracks D (transparent). When C goes low, Q is latched to the last value of D before C went low. Applying S=1 AND R=1 simultaneously when C=1 must be avoided (the NOT gate ensures this never happens in a D latch).

Key Difference: SR Latch has forbidden state (S=R=1). D Latch eliminates this by using D and $\bar{D}$ — so S and R can never be simultaneously 1. The D latch has only ONE data input, making it far simpler and safer to use.

3. Master-Slave Flip-Flops (Pulse-Triggered)

3.1 Introduction & Need for Master-Slave FF

Latches (including gated latches) respond to input changes whenever the enable/clock is HIGH. This property — called TRANSPARENCY — is often undesirable in digital systems. When the output of a flip-flop feeds back to its own input (as in counters), transparency causes the output to change multiple times during a single clock cycle, producing incorrect results.

The purpose of Master-Slave (MS) flip-flops is to PREVENT the flip-flop output from changing or responding to glitches on the input while the clock is active. The output change is delayed until the clock returns to its inactive level.

Key characteristics of Master-Slave (Pulse-Triggered) flip-flops:

- A MS flip-flop consists of two cascaded sections (each a gated latch): 1st section = MASTER, 2nd section = SLAVE.
- Information is entered into the master on one edge/level of the control signal (clock) and transferred to the slave on the NEXT edge/level — i.e., when clock goes from 1 to 0 (falling edge).
- The slave changes state only at the FALLING EDGE of the clock pulse.
- Output changes are thus synchronized to the falling edge of the clock, which is why these are also called PULSE-TRIGGERED flip-flops.
- Each section is a gated SR latch. An inverter between the master and slave clock inputs ensures they are never simultaneously active.

3.2 Master-Slave SR Flip-Flop

The Master-Slave SR flip-flop is constructed using two gated SR latches and one inverter connected in cascade.

Circuit structure:

- MASTER = Gated SR latch with clock C as enable.
- SLAVE = Gated SR latch with clock $\bar{C}$ (inverted) as enable.
- Outputs of Master (Q_M , $\bar{Q}_M$) are connected to S and R inputs of Slave respectively.
- External inputs S and R are applied to the Master only.

Detailed Operation with reference to clock pulse:

Phase 1 – C=0 (Before rising edge, clock LOW): Master is DISABLED (clock=0 → master gated latch disabled). Slave is ENABLED ($\bar{C}=1$ → slave enabled). The state of the master is transferred to the slave — slave takes on master's current state. $Q_S = Q_M$. The external outputs Q and $\bar{Q}$ are the slave outputs.

Phase 2 – C rises from 0→1 (Rising edge): At time t_1 : slave is disabled ($\bar{C}$ goes to 0). Slave retains its last state. At time t_2 : master is enabled ($C=1$). Master responds to S and R inputs and changes state accordingly.

Phase 3 – C=1 (Clock HIGH): Master is enabled and can change state with S and R inputs. But slave is disabled — changes in master are NOT reflected in slave outputs. External outputs remain unchanged.

Phase 4 – C falls from 1→0 (Falling edge): At time t_3 : master is disabled ($C=0$). Master's new state is locked in. At time t_4 : slave is enabled ($\bar{C}=1$). Master's state is transferred to slave. External outputs Q and $\bar{Q}$ change to reflect the new state.

Function Table of Master-Slave SR Flip-Flop:

S	R	C (clock)	Q^+	$\bar{Q}^+$	State
0	0	pulse ↓	Q	$\bar{Q}$	No Change
0	1	pulse ↓	0	1	RESET
1	0	pulse ↓	1	0	SET
1	1	pulse ↓	?	?	Undefined
X	X	0	Q	$\bar{Q}$	No Change

The "↓" symbol in the function table means the output changes on the falling edge of the clock pulse. The "postponed output indicator" symbol ($-$) in the logic symbol indicates that the output changes at the falling edge. When $S=R=1$ during the high clock period, the master enters an undefined state and this is transferred to the slave — hence $S=R=1$ must be avoided.

Since the master's state depends on the period for which clock is at logic-1, and the slave changes at the falling edge, these flip-flops are also called PULSE TRIGGERED FLIP FLOPS.

3.3 Master-Slave JK Flip-Flop (MS-JK)

The Master-Slave JK Flip-Flop is constructed using a MS SR flip-flop with two AND gates added at the input. The AND gates use the flip-flop's own output (Q and $\bar{Q}$) as feedback inputs to generate the S and R signals for the master SR latch.

The main purpose of the MS JK is to overcome the UNDEFINED OUTPUT STATE of the MS SR flip-flop when $S=R=1$. In the JK version, when $J=K=1$, the flip-flop TOGGLES (Q changes to $\bar{Q}$) — a well-defined and useful operation.

Input relationships:

$$S = J \cdot \bar{Q} \quad (\text{J input ANDed with complement output})$$

$$R = K \cdot Q \quad (\text{K input ANDed with normal output})$$

This feedback ensures that when $J=K=1$:

- If $Q=0, \bar{Q}=1$: $S = J \cdot \bar{Q} = 1 \cdot 1 = 1$, $R = K \cdot Q = 1 \cdot 0 = 0 \rightarrow$ Master SETs $\rightarrow Q$ becomes 1 (Toggle)
- If $Q=1, \bar{Q}=0$: $S = J \cdot \bar{Q} = 1 \cdot 0 = 0$, $R = K \cdot Q = 1 \cdot 1 = 1 \rightarrow$ Master RESETs $\rightarrow Q$ becomes 0 (Toggle)

Function Table of Master-Slave JK Flip-Flop:

J	K	C	Q^+	Operation
X	X	0	Q	No Change (clock inactive)
0	0	pulse	Q	No Change (HOLD)
0	1	pulse	0	RESET

J	K	C	Q ⁺	Operation
1	0	pulse	1	SET
1	1	pulse	$\bar{Q}$	TOGGLE (complement)

The JK flip-flop is the most versatile flip-flop. $J=0, K=0 \rightarrow$ Hold; $J=0, K=1 \rightarrow$ Reset; $J=1, K=0 \rightarrow$ Set; $J=1, K=1 \rightarrow$ Toggle. The TOGGLE behavior ($J=K=1$) distinguishes JK from SR — there is NO forbidden/undefined state in JK!

3.4 0s Catching & 1s Catching in MS-JK FF

A Master-Slave JK Flip-Flop is subject to a problem called "0s catching" and "1s catching." This is the primary disadvantage of MS-JK flip-flops and arises because the master is enabled for the entire HIGH period of the clock.

1s Catching (J-catching): If $J=1$ momentarily at any time during the clock HIGH period, the master will SET. Even if J returns to 0, the master retains the SET state (because the slave's feedback keeps master in that state once set). The slave then takes the SET state at the falling clock edge. This is called "1s catching" — a momentary glitch of 1 on J is "caught" and has permanent effect.

0s Catching (K-catching): If the slave latch is in 1-state and the first input to the flip-flop is $J=0, K=1$, the master will RESET. Once reset, even if K returns to 0, the feedback keeps master reset for the rest of the clock high period. The slave resets at falling clock edge. A momentary glitch on K is "caught."

The solution to 0s and 1s catching is to use EDGE-TRIGGERED flip-flops, where J and K inputs must be stable only during a very brief instant around the triggering clock edge, not for the entire clock high period.

3.5 Master-Slave D Flip-Flop

A Master-Slave D Flip-Flop (MSD-FF) is realized by placing an inverter between the S and R inputs of a MS SR flip-flop. This ensures R is always the complement of S , and the single input is called D .

Operation: When clock pulse occurs, D is entered into the master. At the falling edge of the clock, the master state (D value) is transferred to the slave. Thus $Q^+ = D$ after the falling clock edge.

Function Table of MS D Flip-Flop:

D	C	Q ⁺
X	0	Q (no change)
0	pulse ↓	0
1	pulse ↓	1

3.6 Master-Slave T Flip-Flop

A Master-Slave T Flip-Flop (MST-FF) is obtained by shorting both J and K inputs of a MS JK FF together. The combined input is called T (Toggle).

T=0: $J=K=0 \rightarrow$ No change. $Q^+ = Q$ (flip-flop holds its state).

T=1: $J=K=1 \rightarrow$ Toggle. $Q^+ = \bar{Q}$ (flip-flop complements its state).

Function Table of MS T Flip-Flop:

T	C	Q ⁺
X	0	Q (no change)
0	pulse	Q (hold)
1	pulse	$\bar{Q}$ (toggle)

T flip-flop is the simplest for counter design. T=1 at every clock pulse causes the flip-flop to toggle — this is exactly how binary counting works! The LSB toggles every pulse, next bit toggles every 2 pulses, and so on.

4. Edge-Triggered Flip-Flops

Edge-triggered flip-flops solve the 0s and 1s catching problem of master-slave flip-flops. In edge-triggered designs, the flip-flop responds to inputs ONLY at the precise instant of the clock edge (rising or falling). At all other times, input changes have no effect on the output.

Classification of edge-triggered flip-flops:

- Positive Edge-Triggered: State changes on the RISING (0→1) clock edge.
- Negative Edge-Triggered: State changes on the FALLING (1→0) clock edge.

4.1 Positive Edge-Triggered D Flip-Flop

The positive edge-triggered D flip-flop is the most commonly used flip-flop in modern digital design. It is constructed using six NAND gates. D is the data input; C is the clock.

Circuit operation analysis:

When C=0 (clock LOW): NAND gates 2 and 3 outputs are both 1 (regardless of D). The inner $\bar{S}\bar{R}$ latch sees $\bar{S}=\bar{R}=1 \rightarrow$ HOLD. Output remains unchanged regardless of D.

When D=0 and C transitions 0→1 (rising edge): During the positive clock edge, all inputs to NAND gate 3 become 0, making its output = 1, but gate 2 output = 1. The $\bar{S}\bar{R}$ latch gets $\bar{S}=1, \bar{R}=0 \rightarrow$ RESET. Q becomes 0, $\bar{Q}=1$.

When D=1 and C transitions 0→1 (rising edge): Gate 3 output = 0. Gate 2 output = 1. The $\bar{S}\bar{R}$ latch gets $\bar{S}=0, \bar{R}=1 \rightarrow$ SET. Q becomes 1, $\bar{Q}=0$. Also, gate 3 output feeds back to gate 4, making it 1 permanently for the rest of clock-high period — so subsequent D changes have NO EFFECT.

After rising edge (C=1 steady): Due to feedback from gate 3 to gate 4, input changes in D are blocked. The flip-flop is LOCKED in its new state.

Function Table of Positive Edge-Triggered D Flip-Flop:

D	C	Q ⁺
0	↑ (rising)	0
1	↑ (rising)	1
X	0	Q (no change)
X	1	Q (no change)

The ↑ symbol in the clock column indicates a positive (rising) edge trigger. $Q^+ = D$ — the next state equals D, sampled exactly at the rising clock edge.

4.2 Negative Edge-Triggered Flip-Flops

A negative edge-triggered flip-flop changes state on the FALLING (1→0) edge of the clock. It is realized by placing an inverter at the control (clock) input of a positive edge-triggered flip-flop.

When the clock falls from 1 to 0, the inverter's output rises from 0 to 1 — this rising edge triggers the internal positive edge-triggered latch, making the external circuit respond to the falling edge.

Function Table of Negative Edge-Triggered D Flip-Flop:

D	C	Q ⁺
0	↓ (falling)	0
1	↓ (falling)	1
X	0	Q (no change)
X	1	Q (no change)

The ↓ symbol indicates a negative (falling) edge trigger. Logic symbols use a bubble (small circle) at the clock input to indicate negative edge triggering.

4.3 Asynchronous Inputs: PRESET & CLEAR

To provide greater flexibility, many flip-flops incorporate both synchronous and asynchronous inputs. The two primary asynchronous inputs are:

PRESET (PR) — Active LOW: $\overline{PR}$ is an active-low input. A logic-0 applied to $\overline{PR}$ immediately (asynchronously, independent of clock) SETS the flip-flop to 1-state. $\overline{PR}=0$ forces $Q=1$, $\overline{Q}=0$ instantly.

CLEAR (CLR) — Active LOW: $\overline{CLR}$ is an active-low input. A logic-0 applied to $\overline{CLR}$ immediately (asynchronously) RESETS the flip-flop to 0-state. $\overline{CLR}=0$ forces $Q=0$, $\overline{Q}=1$ instantly.

Function Table with Asynchronous Inputs (D flip-flop with $\overline{PR}$ and $\overline{CLR}$):

$\overline{PR}$	$\overline{CLR}$	D	C	Q ⁺	Condition
0	1	X	X	1	PRESET (async SET)
1	0	X	X	0	CLEAR (async RESET)
0	0	X	X	1*	Forbidden (both active)
1	1	0	↑	0	Sync: D=0 sampled
1	1	1	↑	1	Sync: D=1 sampled
1	1	X	0	Q	No change

$\overline{PR}=0$ overrides everything: regardless of D or clock, Q goes to 1. $\overline{CLR}=0$ overrides everything: Q goes to 0. When both $\overline{PR}=0$ and $\overline{CLR}=0$ simultaneously, both Q and $\overline{Q}$ are forced to 1 — this is a FORBIDDEN STATE. When asynchronous inputs are not active ($\overline{PR}=\overline{CLR}=1$), the flip-flop operates normally with synchronous inputs.

Asynchronous inputs are used to INITIALIZE registers and counters to a known state before normal operation begins. At power-up, CLR is pulsed low to reset all flip-flops to 0, ensuring a known starting state for the digital system.

4.4 Positive Edge-Triggered JK Flip-Flop

The positive edge-triggered JK flip-flop is constructed by connecting a positive edge-triggered D flip-flop with a combinational circuit at its input such that $D = J \cdot \overline{Q} + \overline{K} \cdot Q$.

At the rising clock edge, the D input value (determined by J, K, and current Q) is sampled and becomes Q⁺.

Function Table:

J	K	C	Q ⁺	Operation
0	0	↑	Q	Hold
0	1	↑	0	Reset
1	0	↑	1	Set
1	1	↑	$\bar{Q}$	Toggle
X	X	0	Q	No change

This is a positive edge-triggered flip-flop — inputs J and K are sampled ONLY at the rising clock edge. Changes in J and K at other times have NO effect on the output. This eliminates 0s and 1s catching.

4.5 Positive Edge-Triggered T Flip-Flop

Realized by connecting J and K of a positive edge-triggered JK flip-flop together. $T=J=K$.

- $T=0$ at rising edge $\rightarrow Q^+ = Q$ (Hold)
- $T=1$ at rising edge $\rightarrow Q^+ = \bar{Q}$ (Toggle)

The T flip-flop is ideal for counters. Each $T=1$ applied at a clock edge causes the output to toggle, dividing the clock frequency by 2.

4.6 Master-Slave FF with Data Lockout (Edge Triggered MS)

To combine the advantages of master-slave FF (delayed output) with edge-triggering (no 0s/1s catching), a Master-Slave FF with DATA LOCKOUT is used.

In this configuration:

- Master = Positive edge-triggered JK flip-flop (data is entered on positive edge only, no catching).
- Slave = SR latch (state is transferred to slave on negative edge).
- Information enters the master ONLY on the positive clock edge (data locked out during rest of clock cycle).
- Information transfers to slave on the NEGATIVE clock edge.
- This flip-flop is NOT subject to 0s or 1s catching — key advantage over basic MS-JK FF.

Summary of Flip-Flop Types:

- Latch: Level-sensitive, responds whenever enable=1.
- Pulse-Triggered (MS): State captured during clock HIGH, output changes at falling edge. Susceptible to catching.
- Edge-Triggered: Responds ONLY at clock edge. Immune to catching.

Most commonly used in modern design.

5. Characteristic Equations of Flip-Flops

The algebraic description of the next-state (Q^+) of a flip-flop in terms of its current inputs and present state (Q) is called its CHARACTERISTIC EQUATION. It is derived by constructing a K-map for Q^+ from the next-state table (also called the function/characteristic table).

The next-state table is derived from the function table by considering all combinations of inputs AND the present state Q together.

5.1 SR Flip-Flop Characteristic Equation

Next State Table for SR Flip-Flop:

S	R	Q (present)	Q ⁺ (next)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

K-Map for Q^+ (variables S, RQ grouped as 2-variable K-map with S on rows, RQ on columns):

After plotting and grouping (using X as don't care):

$$Q^+ = S + R' \cdot Q$$

Constraint: $S \cdot R = 0$ (S and R cannot both be 1 simultaneously — forbidden state condition)

SR FF Characteristic Equation: $Q^+ = S + R' \cdot Q$, with constraint $SR = 0$

5.2 JK Flip-Flop Characteristic Equation

Next State Table for JK Flip-Flop:

J	K	Q (present)	Q ⁺ (next)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1

J	K	Q (present)	Q ⁺ (next)
1	0	1	1
1	1	0	1
1	1	1	0

K-Map for Q⁺ (J on rows, KQ on columns — 3 variable K-map):

Group 1: {J=1,K=0,Q=0}, {J=1,K=0,Q=1}, {J=1,K=1,Q=0} → J·K̄ + J·Q̄... Simplified:

$$Q^+ = J \cdot \bar{Q} + \bar{K} \cdot Q$$

JK FF Characteristic Equation: $Q^+ = J \cdot \bar{Q} + \bar{K} \cdot Q$ This is the MOST IMPORTANT equation — memorize it! It shows:

- J=1,K=0: $Q^+ = Q \cdot 1 + 1 \cdot \bar{Q} = Q + \bar{Q} = 1$ (SET) ✓
- J=0,K=1: $Q^+ = 0 + 0 = 0$ (RESET) ✓
- J=K=0: $Q^+ = 0 + Q = Q$ (HOLD) ✓
- J=K=1: $Q^+ = \bar{Q} + 0 = \bar{Q}$ (TOGGLE) ✓

5.3 D Flip-Flop Characteristic Equation

The D flip-flop is the simplest of all. Its next-state table:

D	Q (present)	Q ⁺ (next)
0	0	0
0	1	0
1	0	1
1	1	1

K-Map: Q⁺ depends only on D, regardless of Q. The entire Q=0 column is 0 and the Q=1 column is 1 (for D=1). Result:

$$Q^+ = D$$

D FF Characteristic Equation: $Q^+ = D$ The simplest possible equation — next state equals D input. No constraint needed.

5.4 T Flip-Flop Characteristic Equation

The T flip-flop next-state table:

T	Q (present)	Q ⁺ (next)
0	0	0
0	1	1
1	0	1
1	1	0

K-Map: Q⁺=1 when T=1,Q=0 and when T=0,Q=1. This is an XOR pattern:

$$Q^+ = T \oplus Q \quad (T \text{ XOR } Q)$$

Alternatively: $Q^+ = T \cdot \bar{Q} + \bar{T} \cdot Q$

T FF Characteristic Equation: $Q^+ = T \oplus Q$ • T=0: $Q^+ = 0 \oplus Q = Q$ (Hold)• T=1: $Q^+ = 1 \oplus Q = \bar{Q}$ (Toggle)

5.5 Excitation Tables of All Flip-Flops

An Excitation Table is the INVERSE of the characteristic/function table. Instead of asking "given inputs, what is Q^+ ?", it asks: "given a required transition from Q to Q^+ , what INPUT is needed?" Excitation tables are essential for COUNTER DESIGN using the State Table approach.

Transition $Q \rightarrow Q^+$	SR FF (S, R)	JK FF (J, K)	D FF (D)
$0 \rightarrow 0$	S=0, R=X	J=0, K=X	D=0
$0 \rightarrow 1$	S=1, R=0	J=1, K=X	D=1
$1 \rightarrow 0$	S=0, R=1	J=X, K=1	D=0
$1 \rightarrow 1$	S=X, R=0	J=X, K=0	D=1

Transition $Q \rightarrow Q^+$	T FF (T)
$0 \rightarrow 0$	T=0
$0 \rightarrow 1$	T=1
$1 \rightarrow 0$	T=1
$1 \rightarrow 1$	T=0

Interpretation of X (don't care) in excitation table: For a $0 \rightarrow 0$ transition in SR FF, S must be 0 (cannot set), but R can be anything (0 or 1) — hence R=X. This gives freedom to simplify K-maps during counter design.

Memorize: JK FF excitation table has the most X (don't cares) → gives maximum simplification in K-maps → JK FF gives the simplest counter designs. D FF excitation table has NO X → D always equals Q^+ .

vtu-easy.com

6. Registers

6.1 Introduction to Registers & Shift Registers

Registers and counters are the two most important applications of clocked flip-flops. A REGISTER is a collection of flip-flops connected in cascade (one after another), used to store, move, and manipulate binary data.

Key facts about registers:

- A collection of flip-flops in cascade is called a REGISTER.
- Each flip-flop stores one bit. An n-bit register has n flip-flops and can store n bits.
- Each combination of 1s and 0s stored in a register is called a STATE or CONTENT.
- A 4-bit register has 16 possible states (0000 to 1111). It stores a NIBBLE of data.
- Registers are used to store data in a digital system to make it available to logic elements during the computing process.

Shift Register: A register capable of MOVING (shifting) data stored in its flip-flops from one position to another on each clock pulse is called a shift register. Shift registers that can shift in BOTH directions are called bidirectional; those in only one direction are unidirectional.

Classification of shift registers based on data input/output:

Type	Full Name	Description
SISO	Serial In Serial Out	Data enters bit-by-bit; exits bit-by-bit
SIPO	Serial In Parallel Out	Data enters serially; all bits available simultaneously
PIPO	Parallel In Parallel Out	All bits loaded simultaneously; available simultaneously
PISO	Parallel In Serial Out	All bits loaded simultaneously; exits bit-by-bit

6.2 4-Bit SISO (Serial In Serial Out) Shift Register

A 4-bit SISO unidirectional shift register is constructed using four positive edge-triggered D flip-flops connected in cascade.

Circuit description:

- Q output of each flip-flop is connected to D input of the NEXT flip-flop.
- All clock inputs are connected to a common clock signal.
- Serial data enters at D input of the FIRST (leftmost) flip-flop.
- Serial data exits from Q output of the LAST (rightmost) flip-flop.

Operation:

- At each positive clock edge, the content of each flip-flop is shifted ONE POSITION to the RIGHT.
- The new bit from serial input enters the leftmost flip-flop.
- The bit from the rightmost flip-flop is lost (serial output).

Example: If register content = 0011 and serial input = 1:

- After 1 clock pulse: content = 1001 (1 entered left, leftmost 0 shifted out... wait: shift right means $MSB \leftarrow \text{serial}$, $Q_0 \rightarrow Q_1 \rightarrow Q_2 \rightarrow Q_3$, Q_3 exits). Content: new $MSB = \text{serial}=1$, rest shift right. 0011 $\rightarrow$ 1001.
- The serial data input appears at the serial data output after 4 clock pulses (one full register length).

Circular Shift Register: If the serial-data output (Q of last FF) is connected back to the serial-data input (D of first FF), data circulates indefinitely. This is called a CIRCULAR SHIFT REGISTER. Example: 1001 $\rightarrow$ after one clock pulse with Q_3 feedback $\rightarrow$ 1100 $\rightarrow$ 0110 $\rightarrow$ 0011 $\rightarrow$ 1001 (original).

SISO register stores 4 bits but requires 4 clock pulses to input 4 bits and another 4 to extract them. Total: 8 clock cycles for a 4-bit transfer. This is slower than parallel but uses only 1 wire for data transfer — useful in serial communication.

6.3 4-Bit SIPO (Serial In Parallel Out) Shift Register

A 4-bit SIPO shift register is identical in construction to SISO, but provides parallel outputs from ALL flip-flops simultaneously.

Circuit description:

- Constructed using 4 positive edge-triggered D flip-flops in cascade (same as SISO).
- Outputs Q_A , Q_B , Q_C , Q_D are all accessible externally simultaneously (parallel output).
- Serial data enters from the left; after 4 clock pulses, the 4-bit word is available in parallel.

Operation: Data is entered serially, one bit per clock pulse. After n clock pulses ($n=4$ for 4-bit register), the full n -bit word is simultaneously available at all Q outputs. This is the basis of SERIAL-TO-PARALLEL CONVERSION — a fundamental operation in communication systems (e.g., converting serial UART data to parallel for a processor).

6.4 4-Bit Parallel-In Shift Register (PISO/PIPO)

The 4-bit Parallel-In shift register uses an additional control line — the LOAD/SHIFT line — to select between two modes of operation.

Circuit description: Each stage uses a 2-input multiplexer (or AND-OR logic) with the LOAD/SHIFT control:

LOAD/SHIFT = 0 (Parallel Load mode): Lower AND gates are enabled. Parallel data inputs I_A , I_B , I_C , I_D are transferred into the flip-flops simultaneously on the next positive clock edge. (Like PIPO operation.)

LOAD/SHIFT = 1 (Shift mode): Upper AND gates are enabled. The register operates as a unidirectional SISO (or SIPO) shift register, shifting data right one position per clock pulse.

This register can also be configured as SISO (shift with serial output) and SIPO (shift with parallel output). It is very flexible — one register implements multiple functions.

Application: PISO (Parallel-In Serial-Out) is the basis of PARALLEL-TO-SERIAL CONVERSION — loading a word in parallel and transmitting it serially. Used in communication interfaces (SPI, UART transmission side).

6.5 Bidirectional / Universal Shift Register

A universal shift register is capable of shifting contents in BOTH DIRECTIONS (left and right) and also supports parallel load. It is the most general-purpose shift register.

Construction: Each stage uses a 4:1 MUX (4-input multiplexer) with two mode control signals S_1 and S_0 . The Mode Selection Table determines operation:

S_1	S_0	Data Line Selected	Operation
0	0	I_0 (own Q output)	HOLD – data recirculates, no shift
0	1	I_1 (left neighbor Q)	SHIFT RIGHT – data moves right
1	0	I_2 (right neighbor Q)	SHIFT LEFT – data moves left
1	1	I_3 (parallel input)	PARALLEL LOAD – load data

Additional features:

- SIR (Serial Input Right): Serial input for shift-right operation (applied to leftmost stage).
- SIL (Serial Input Left): Serial input for shift-left operation (applied to rightmost stage).
- CLEAR (active-low): Clears all flip-flops asynchronously to 0 independent of clock.
- All operations occur on the positive clock edge since positive edge-triggered D flip-flops are used.

Universal Shift Register is used in: Serial-parallel and parallel-serial converters, Arithmetic shift operations (shift left = multiply by 2, shift right = divide by 2), Sequence generators, and Data storage/transfer in processors.

7. Counters

7.1 Introduction & Terminology

A counter is a collection of flip-flops configured to produce a specified OUTPUT PATTERN SEQUENCE in response to clock pulses. A counter is therefore also called a PATTERN GENERATOR — each clock pulse causes the counter to advance to the next state in its sequence.

State: Each output sequence value stored in the flip-flops is called a STATE of the counter.

Modulus: The total number of states in the counting sequence is called the MODULUS (or MOD) of the counter. A counter with m states is called a mod- m counter. Example: A counter cycling through 10 states (0–9) is a Mod-10 counter.

Binary Counter: Outputs a binary number sequence. n flip-flops can configure a counter up to modulus 2^n .

Up Counter: Counts from 0 to (2^n-1) and repeats.

Down Counter: Counts from (2^n-1) to 0 and repeats.

State diagram: A state diagram is used to describe the operation of a counter. It shows each state as a circle and transitions as arrows between circles.

Types of counters:

- **ASYNCHRONOUS (Ripple) Counter:** Clock is applied to only the first flip-flop. Subsequent FFs are clocked by $\bar{Q}$ output of the previous FF. State changes ripple through — not simultaneous.
- **SYNCHRONOUS Counter:** Clock is applied simultaneously to ALL flip-flops. All state changes occur at the same time — no ripple delay.

7.2 4-Bit Binary Ripple Counter (Asynchronous Counter)

A 4-bit binary ripple UP counter is configured using four POSITIVE edge-triggered T flip-flops ($T=1$ always, so each FF toggles on every triggering edge it receives).

Circuit configuration:

- FF_0 (Q_0 , LSB): $T=1$, Clock = external count clock. Q_0 toggles on every positive edge of the clock.
- FF_1 (Q_1): Clock = $\bar{Q}_0$ output of FF_0 . Q_1 toggles when $\bar{Q}_0$ has a rising edge, i.e., when Q_0 falls (0→1 transition of Q_0).
- FF_2 (Q_2): Clock = $\bar{Q}_1$. Toggles when Q_1 falls.
- FF_3 (Q_3 , MSB): Clock = $\bar{Q}_2$. Toggles when Q_2 falls.

Counting sequence (4-bit binary up count):

Count	Q_3	Q_2	Q_1	Q_0	Decimal
Initial	0	0	0	0	0
After CP1	0	0	0	1	1
After CP2	0	0	1	0	2
After CP3	0	0	1	1	3
After CP4	0	1	0	0	4

Count	Q ₃	Q ₂	Q ₁	Q ₀	Decimal
After CP8	1	0	0	0	8
After CP15	1	1	1	1	15
After CP16	0	0	0	0	0 (repeats)

Key characteristics of Ripple Counters:

- Called RIPPLE counter because a state change in the first FF "ripples" through subsequent FFs with propagation delay.
- Also called ASYNCHRONOUS counter — clock is not applied simultaneously to all FFs.
- WORST CASE SETTLING TIME = $n \times t_{pd}$, where n = number of stages, t_{pd} = propagation delay of one FF. For 4-bit: settling time = $4 \times t_{pd}$.
- Advantage: Very simple circuit — requires minimum hardware.
- Disadvantage: Speed limited by propagation delay; intermediate glitch states appear during counting transitions.

For a binary DOWN ripple counter: Connect the clock of each subsequent FF to the Q output (not $\bar{Q}$) of the previous FF. The counter counts $1111 \rightarrow 1110 \rightarrow \dots \rightarrow 0000 \rightarrow 1111$.

7.3 Synchronous Binary Counter

In a synchronous counter, the clock pulse is applied DIRECTLY AND SIMULTANEOUSLY to the control (clock) inputs of ALL flip-flops. This eliminates the propagation delay problem of ripple counters.

Design principle for 4-bit synchronous up counter using T flip-flops:

Observation from the counting sequence:

- Q₀ TOGGLES on EVERY clock pulse $\rightarrow T_0 = 1$ always.
- Q₁ toggles whenever Q₀ = 1 $\rightarrow T_1 = Q_0$.
- Q₂ toggles whenever Q₀=1 AND Q₁=1 $\rightarrow T_2 = Q_0 \cdot Q_1$.
- Q₃ toggles whenever Q₀=Q₁=Q₂=1 $\rightarrow T_3 = Q_0 \cdot Q_1 \cdot Q_2$.

Circuit: AND gates compute the T inputs:

$$T_0 = \text{Count_Enable (CE)}$$

$$T_1 = \text{CE} \cdot Q_0$$

$$T_2 = \text{CE} \cdot Q_0 \cdot Q_1$$

$$T_3 = \text{CE} \cdot Q_0 \cdot Q_1 \cdot Q_2$$

Problem with this design: As the number of stages increases, the number of inputs to the AND gate also increases. For n stages, the AND gate for FF_{n-1} needs n inputs. This limits speed due to large AND gate delays.

Modified design (serial gating): To keep AND gates at 2-inputs, use cascaded 2-input AND gates:

$$T_0 = \text{CE}$$

$$T_1 = T_0 \cdot Q_0$$

$$T_2 = T_1 \cdot Q_1$$

$$T_3 = T_2 \cdot Q_2$$

This is called SERIAL GATING — the enable ripples through AND gates. While the AND gates still have delay, this is significantly less than FF settling time.

7.4 4-Bit Synchronous Counter with Parallel Load

A practical 4-bit synchronous binary counter IC (like 74163) includes additional features:

- **PARALLEL LOAD:** A logic-1 at the LOAD input causes the counter to load data D_0, D_1, D_2, D_3 into the flip-flops on the next clock edge (instead of counting).
- **COUNT ENABLE:** A logic-1 on the count enable input allows the counter to count; logic-0 holds the count.
- **CARRY OUTPUT (C_0):** Goes HIGH when count reaches maximum (1111 for 4-bit). Used to cascade multiple counters to form higher-bit counters.

The carry output C_0 is connected to the count enable of the next stage to create 8-bit, 12-bit, or 16-bit counters from 4-bit modules.

Ripple Counter vs Synchronous Counter: • Ripple: Simple, but slow ($n \times t_{pd}$ settling time), has glitch states. • Synchronous: All FFs triggered simultaneously, no ripple delay, no glitches. Settling time = t_{pd} of ONE FF + AND gate delay. Preferred for high-speed designs.

vtu-easy.com

8. Counters Based on Shift Registers

The overall operation of a digital system is often divided into a sequence of time periods. Special counters based on shift registers provide timing sequences with unique decoding properties. Two important types are: (1) Ring Counter and (2) Johnson Counter.

8.1 Ring Counter

A ring counter is a CIRCULAR SHIFT REGISTER initialized (preset) so that ONLY ONE flip-flop is in the 1-state while ALL OTHERS are in the 0-state. Upon each clock pulse, the single 1-bit "rings" (circulates) around the register.

Key properties:

- An n-flip-flop ring counter has exactly n STATES (not 2^n like a binary counter). Hence its modulus = n.
- It provides DECODED output — only one FF is HIGH at any time, directly representing the count. No external decoder needed.
- Initial state must be loaded using PRESET/CLEAR inputs: typically $Q_A=1, Q_B=Q_C=Q_D=0$ → state = 1000.

4-bit Ring Counter counting sequence (initialized to 1000):

State	Q_A	Q_B	Q_C	Q_D	Representations
0	1	0	0	0	Time slot 0
1	0	1	0	0	Time slot 1
2	0	0	1	0	Time slot 2
3	0	0	0	1	Time slot 3
4	1	0	0	0	Repeats (back to state 0)

Circuit: Q output of last FF connects back to D input of first FF. All FFs share the same clock. The 1 circulates continuously. Ring counter produces a decoded output — each output identifies exactly one unique time period.

Disadvantage: n flip-flops give only n states — very inefficient use of hardware. A 4-bit binary counter gives 16 states with 4 FFs; a 4-bit ring counter gives only 4 states.

Advantage: No external decoder needed; each FF output directly identifies a time slot. Used in timing and sequencing circuits.

8.2 Johnson Counter (Twisted Ring / Switch-Tail Counter)

A Johnson counter (also called a Twisted Ring Counter or Switch-Tail Counter) is a variation of the ring counter where the $\bar{Q}$ output of the LAST flip-flop is fed back to the D input of the FIRST flip-flop (instead of Q as in ring counter).

Key properties:

- An n-flip-flop Johnson counter has $2n$ STATES (twice as many as a ring counter). Hence modulus = $2n$.
- A 4-flip-flop Johnson counter has modulus 8 (Mod-8).
- Decoding requires only 2-input AND gates (simpler than binary counter decoding).
- The sequence is self-starting if designed carefully, but may need initialization.

4-flip-flop (Mod-8) Johnson Counter counting sequence:

Clock	Q_A	Q_B	Q_C	Q_D	State
Init	0	0	0	0	State 0
CP1	1	0	0	0	State 1
CP2	1	1	0	0	State 2
CP3	1	1	1	0	State 3
CP4	1	1	1	1	State 4
CP5	0	1	1	1	State 5
CP6	0	0	1	1	State 6
CP7	0	0	0	1	State 7
CP8	0	0	0	0	Returns to State 0

Pattern: First half of sequence: 0s replaced by 1s starting from left. Second half: 1s replaced by 0s starting from left. The pattern is called "walking ones" then "walking zeros."

Decoding logic: Each of the $2n$ states can be decoded using a 2-input AND gate or AND gate with inversion:

- State 0 (0000): $Q_A \cdot Q_D$ (both ends 0)
- State 1 (1000): $Q_A \cdot \bar{Q}_B$ ($Q_A=1$, $Q_B=0$)
- State 4 (1111): $Q_A \cdot Q_D$ (both ends 1)
- And so on — each state identified by adjacent pair of differing bits.

Comparison: Ring vs Johnson:

Parameter	Ring Counter	Johnson Counter
States (n FFs)	n states	$2n$ states
Feedback	$Q_{last} \rightarrow D_{first}$	$\bar{Q}_{last} \rightarrow D_{first}$
Decoding	Direct (no gates)	2-input AND gates
4-bit modulus	Mod-4	Mod-8

Johnson Counter is preferred over Ring Counter when more states are needed with

Parameter	Ring Counter	Johnson Counter
the same hardware. Johnson doubles the state count. Both are used as timing/sequence generators where decoded outputs are needed directly without a binary decoder.		

vtu-easy.com

9. Design of Synchronous mod-n Counters

The design of synchronous counters is one of the most important and frequently examined topics in VTU Digital Systems Design. The procedure involves determining the required flip-flop inputs using excitation tables and K-maps, then implementing the counter with gates.

9.1 General Design Procedure

Follow these 6 steps for designing ANY synchronous mod-n counter using any flip-flop type:

1. **OBTAIN THE STATE DIAGRAM:** Draw the state diagram showing the counting sequence (n states in order, with arrow from last back to first).
2. **DETERMINE NUMBER OF FLIP-FLOPS:** The number of flip-flops required is $\lceil \log_2(n) \rceil$ — the ceiling of $\log_2(n)$. For example, mod-6: $\lceil \log_2(6) \rceil = \lceil 2.58 \rceil = 3$ flip-flops.
3. **WRITE THE STATE TABLE (EXCITATION TABLE):** Create a table with columns for present state (Q values), next state (Q^+ values), and the required flip-flop inputs (using the excitation table for the chosen FF type).
4. **DERIVE K-MAPS:** For each flip-flop input, draw a K-map using the present state variables as map variables. Use don't-cares (X) for the unused states (states beyond n-1 in the 2^m states available).
5. **MINIMIZE & EXTRACT BOOLEAN EXPRESSIONS:** Simplify each K-map to get the Boolean expression for each flip-flop input.
6. **DRAW THE CIRCUIT:** Implement the flip-flops with their computed input expressions using logic gates.

IMPORTANT: Unused states (above the counter modulus) can be assigned as don't cares in K-maps, giving maximum simplification. However, always verify that the counter does not lock up in an unused state — if it can, add logic to force it back to the counting sequence.

9.2 Design of Mod-6 Synchronous Counter using T Flip-Flop

Problem: Design a synchronous Mod-6 (count 0–5, then repeat) UP counter using T flip-flops.

Step 1: Number of FFs = $\lceil \log_2(6) \rceil = 3$ flip-flops: Q_A (MSB), Q_B, Q_C (LSB).

Step 2: State sequence: 000→001→010→011→100→101→000 (states 110 and 111 are unused).

Step 3: State/Excitation Table (T FF: T=0 if Q→Q same; T=1 if Q→Q changes):

Q_A	Q_B	Q_C	Q_A ⁺	Q_B ⁺	Q_C ⁺	T_A	T_B	T_C
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	0	0	0	1	0	1
1	1	0	X	X	X	X	X	X

Q_A	Q_B	Q_C	Q_A ⁺	Q_B ⁺	Q_C ⁺	T_A	T_B	T_C
1	1	1	X	X	X	X	X	X

Step 4: K-Maps for T_A, T_B, T_C (3-variable, Q_A on one axis, Q_B Q_C on other with Gray code):

K-map for T_C: T_C = 1 for all 6 valid states. Group all → T_C = 1 (always HIGH — Q_C toggles every clock pulse).

K-map for T_B: T_B = 1 at states 001(Q_B=0, Q_C=1) and 011(Q_B=1, Q_C=1). Group → T_B = Q_C.

K-map for T_A: T_A = 1 at states 011 and 101. Grouping with don't cares: T_A = Q_B · Q_C + Q_A · Q_C = Q_C · (Q_A + Q_B). After further simplification: T_A = Q_A ⊕ Q_B when Q_C=1... checking: Q_A=0, Q_B=1, Q_C=1 → T_A=1 ✓; Q_A=1, Q_B=0, Q_C=1 → T_A=1 ✓. So T_A = Q_B · Q_C + Q_A · Q_C' ... Using don't cares: T_A = Q_B · Q_C (covers 011 → T_A=1) and Q_A · Q_C (covers 101 → T_A=1). T_A = Q_C(Q_A + Q_B).

Step 5: Final expressions:

$$\begin{aligned}T_C &= 1 \\T_B &= Q_C \\T_A &= Q_C \cdot (Q_A + Q_B)\end{aligned}$$

Step 6: Implementation: T_C input tied HIGH. T_B = Q_C (direct wire). T_A = 3-input gate: Q_C AND (Q_A OR Q_B). Counter works correctly through states 0–5 and resets.

9.3 Design of Mod-6 Synchronous Counter using JK Flip-Flop

Problem: Design a synchronous Mod-6 UP counter using JK flip-flops.

Step 1: 3 flip-flops: Q_A (MSB), Q_B, Q_C (LSB). Same state sequence as above.

Step 2: Excitation table for JK FF:

Use JK excitation table: 0→0: J=0, K=X; 0→1: J=1, K=X; 1→0: J=X, K=1; 1→1: J=X, K=0.

Q_A	Q_B	Q_C	Q_A ⁺	Q_B ⁺	Q_C ⁺	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	0	0	0	X	1	0	X	X	1
1	1	0	X	X	X	X	X	X	X	X	X
1	1	1	X	X	X	X	X	X	X	X	X

Step 3: K-map minimization for each J and K input (3-variable maps with Q_A, Q_B, Q_C):

J_A K-map: J_A=1 at 011(Q_A=0, Q_B=1, Q_C=1), X at 110, 111. Group J_A=1+X: J_A = Q_B · Q_C.

K_A K-map: K_A=1 at 101(Q_A=1, Q_B=0, Q_C=1), X at 110, 111. K_A = Q_C (with don't cares). Check: K_A = Q_B' · Q_C.

J_B K-map: J_B=1 at 001 and 100(after don't care). Group: J_B = Q_C + Q_A · Q_B'.

K_B K-map: K_B=1 at 011 and 111(X). K_B = Q_C.

J_C K-map: $J_C=1$ at states 000,010,100 (all $Q_C=0$ valid states). $J_C = Q_A'Q_B' + Q_A'Q_B + Q_A \cdot Q_B' = Q_B' + Q_A' = (Q_A \cdot Q_B)'$. Simplify: $J_C = 1$ (check: all valid 0-states have $J_C=1$). $J_C = 1$.

K_C K-map: $K_C=1$ at states 001,011,101 (all $Q_C=1$ valid). $K_C = 1$.

Step 4: Final simplified expressions (JK counter):

$$\begin{aligned} J_A &= Q_B \cdot Q_C & K_A &= Q_B' \cdot Q_C \text{ (or } Q_A' \cdot Q_C \text{ using don't cares)} \\ J_B &= Q_C & K_B &= Q_C \\ J_C &= 1 & K_C &= 1 \end{aligned}$$

Note: JK FF gives many X don't cares → greatly simplifies the expressions compared to other FF types.

9.4 Design using D Flip-Flop

For D flip-flop: $D = Q^+$ (next state directly). No separate excitation table computation needed — D input for each FF equals the required next-state value.

Step 1: 3 FFs. Same mod-6 state table.

Step 2: $D_A = Q_A^+$, $D_B = Q_B^+$, $D_C = Q_C^+$ from the state table.

Q_A	Q_B	Q_C	Q_A^+	Q_B^+	Q_C^+	D_A	D_B	D_C
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	0
0	1	0	0	1	1	0	1	1
0	1	1	1	0	0	1	0	0
1	0	0	1	0	1	1	0	1
1	0	1	0	0	0	0	0	0
1	1	X	X	X	X	X	X	X

Step 3: K-map minimization (3 variable, with X for states 110,111):

$$D_C = Q_A' \cdot Q_C' + Q_B \cdot Q_C' + Q_A \cdot Q_B' = Q_C' \text{ (using don't cares and grouping) } \dots$$

After careful K-map analysis:

$$\begin{aligned} D_A &= Q_B \cdot Q_C + Q_A \cdot Q_C' & D_B &= Q_B' \cdot Q_C + Q_B \cdot Q_C' & D_C &= Q_A' \cdot Q_C' + Q_B' \cdot Q_C' \end{aligned}$$

D FF requires more complex combinational logic than T or JK because there are no don't cares in the D excitation table. $D = Q^+$ exactly.

9.5 Design using SR Flip-Flop

SR excitation table: $0 \rightarrow 0$: $S=0, R=X$; $0 \rightarrow 1$: $S=1, R=0$; $1 \rightarrow 0$: $S=0, R=1$; $1 \rightarrow 1$: $S=X, R=0$.

The same mod-6 state table is used. For each transition, determine S and R for each FF:

Q_A	Q_B	Q_C	Q_A^+	Q_B^+	Q_C^+	S_A	R_A	S_B	R_B	S_C	R_C	
0	0	0	0	0	1	0	X	0	X	1	0	
0	0	1	0	1	0	0	X	1	0	0	1	

Q_A	Q_B	Q_C	Q_A ⁺	Q_B ⁺	Q_C ⁺	S_A	R_A	S_B	R_B	S_C	R_C	
0	1	0	0	1	1	0	X	X	0	1	0	
0	1	1	1	0	0	1	0	0	1	0	1	
1	0	0	1	0	1	X	0	0	X	1	0	
1	0	1	0	0	0	0	1	0	X	0	1	
1	1	X	X	X	X	X	X	X	X	X	X	

K-map minimization gives (with don't cares for unused states):

$$\begin{aligned}
 S_A &= Q_B \cdot Q_C & R_A &= Q_A \cdot Q_C \\
 S_B &= Q_B' \cdot Q_C & R_B &= Q_B \cdot Q_C \\
 S_C &= Q_C' & R_C &= Q_C
 \end{aligned}$$

SR FF expressions are simpler than D FF but more complex than JK FF (due to fewer X don't cares than JK).

Design Complexity Comparison (for mod-n counters): T FF: Simplest expressions for binary counters. Best for ripple counters. JK FF: Most don't cares → simplest for general mod-n synchronous counters. D FF: $D = Q^*$ directly, but no don't cares → most gates needed. SR FF: Intermediate complexity. Constraint $SR=0$ must be verified. For VTU exams: JK FF gives the minimum hardware and is usually the preferred choice for synchronous counter design problems.

10. Quick Revision – Exam Ready Summary

This section provides a consolidated, rapid-revision reference of all key concepts, equations, tables, and patterns for VTU BEC302 Module 3 examinations.

10.1 Key Definitions

Term	Definition
Flip-Flop	Bistable device with inputs. Stores 1 bit. Changes state only on clock edge (edge-triggered) or falling edge after pulse (MS). Retains state until triggered.
Latch	Level-sensitive bistable device. Changes state when enable=HIGH. Transparent when enabled. Examples: SR, $\bar{S}\bar{R}$, Gated SR, D latch.
Master-Slave FF	Two cascaded gated latches (master + slave + inverter). Master captures on rising edge; slave transfers on falling edge. Output changes at falling clock edge.
Edge-Triggered FF	Responds ONLY at clock edge ($\uparrow$ or $\downarrow$). Immune to 0s/1s catching. Most used in modern design.
Characteristic Eqn	Algebraic expression for Q^+ in terms of inputs and present state Q . Derived from next-state table via K-map.
Excitation Table	Inverse of function table. Given transition $Q \rightarrow Q^+$, what input is required? Used in counter design.
Register	Collection of n flip-flops in cascade. Stores n bits. Can be shift register (SISO, SIPO, PISO, PIPO, Universal).
Shift Register	Register that moves data left or right one position per clock pulse. Unidirectional or bidirectional.
Counter	Flip-flops configured to produce a specified output sequence. Classified as ripple (async) or synchronous. Modulus = total number of states.
Ring Counter	Circular shift register with ONE 1-bit circulating. n FFs $\rightarrow n$ states. Decoded output — no decoder needed.
Johnson Counter	Twisted ring counter ($\bar{Q}_{last} \rightarrow D_{first}$). n FFs $\rightarrow 2n$ states. Decoded with 2-input AND gates.
Modulus (MOD)	Total number of states in counting sequence. Mod-6 counter cycles through 6 states.
0s/1s Catching	MS-JK problem: momentary $J=1$ (or $K=1$) during clock-HIGH is "caught" and has permanent effect. Solved by edge triggering.
Metastable State	Third equilibrium state of bistable element where both outputs are midway between 0 and 1. Unpredictable — must be avoided.
PRESET / CLEAR	Asynchronous inputs. $\bar{P}\bar{R}=0$ forces $Q=1$ instantly (Set). $\bar{C}\bar{L}\bar{R}=0$ forces $Q=0$ instantly (Reset). Independent of clock.

10.2 Characteristic Equations Quick Reference

Flip-Flop	Characteristic Equation	Notes & Constraint
SR FF	$Q^+ = S + \bar{R} \cdot Q$	Constraint: $S \cdot R = 0$ (forbidden state $SR=11$)
JK FF	$Q^+ = J \cdot \bar{Q} + \bar{K} \cdot Q$	No forbidden state. $J=K=1 \rightarrow$ Toggle ($\bar{Q}$).
D FF	$Q^+ = D$	Simplest. No constraint. Q^+ always = D.
T FF	$Q^+ = T \oplus Q$	$T=0$: Hold. $T=1$: Toggle. Used in counters.

10.3 Excitation Tables

$Q \rightarrow Q^+$	SR (S,R)	JK (J,K)	D	T
$0 \rightarrow 0$	0, X	0, X	0	0
$0 \rightarrow 1$	1, 0	1, X	1	1
$1 \rightarrow 0$	0, 1	X, 1	0	1
$1 \rightarrow 1$	X, 0	X, 0	1	0

10.4 Flip-Flop Comparison Table

Parameter	SR FF	JK FF	D FF	T FF
Inputs	S, R	J, K	D	T
Forbidden	$SR=11$	None	None	None
Toggle	No	$J=K=1$	No	$T=1$
Char. Eqn	$S + \bar{R}Q$	$J\bar{Q} + \bar{K}Q$	D	$T \oplus Q$
X in Excitation	1 per row	2 per row (max)	0	0
Counter Design	Moderate	Simplest	Most complex	Binary counters
Applications	FSMs	Universal	Shift regs, D-latches	Counters, dividers

10.5 Counter Design Steps & VTU Exam Patterns

6-Step Counter Design Procedure (Memorize!)

- Obtain state diagram from counting sequence.
- Find no. of FFs: $n = \lceil \log_2(\text{modulus}) \rceil$.
- Write excitation table using chosen FF's excitation entries.
- Draw K-maps for each FF input; use X for unused states.
- Minimize K-maps $\rightarrow$ Boolean expressions.
- Draw circuit with FFs and gates from the expressions.

Latch vs FF Quick Comparison

Property	Latch	Flip-Flop
Sensitivity	Level-sensitive	Edge-sensitive
When changes	Enable=HIGH (transparent)	Only at clock edge

Property	Latch	Flip-Flop
Catching problem	Yes	No (edge-triggered)
Use case	Temporary data hold	Registers, counters, FSMs

Common VTU Questions & Mark Weightage

VTU Question Pattern	Marks
Explain Master-Slave SR/JK FF with circuit, function table & timing diagram.	10 M
Explain 0s and 1s catching in MS-JK FF. How is it overcome?	5 M
Derive characteristic equation of SR/JK/D/T FF from next-state table and K-map.	5–10 M
Draw and explain 4-bit SISO/SIPO/Universal shift register with timing diagram.	10 M
Explain Ring Counter and Johnson Counter with state sequence and logic diagram.	10 M
Design synchronous mod-n counter using T/JK/D/SR FF (state table + K-maps + circuit).	10 M
Draw 4-bit synchronous/ripple binary counter. Write counting sequence.	5–10 M
Write excitation tables of all 4 flip-flop types and explain how to use them.	5 M
Compare ripple counter vs synchronous counter (advantages, disadvantages).	5 M
Explain positive and negative edge-triggered D FF with logic diagram and function table.	10 M

Formula Quick-Reference Card

CHARACTERISTIC EQUATIONS: SR: $Q^+ = S + \bar{R} \cdot Q$ (SR=0) | JK: $Q^+ = J \cdot \bar{Q} + \bar{K} \cdot Q$ | D: $Q^+ = D$ | T: $Q^+ = T \oplus Q$ EXCITATION (for counter design): 0→0: SR(0,X), JK(0,X), D=0, T=0 0→1: SR(1,0), JK(1,X), D=1, T=1 1→0: SR(0,1), JK(X,1), D=0, T=1 1→1: SR(X,0), JK(X,0), D=1, T=0

COUNTER FACTS: Ripple counter settling time = $n \times t_{pd}$ | No. of FFs = $\lceil \log_2(\text{mod}) \rceil$ Ring counter: n FFs → n states | Johnson counter: n FFs → 2n states Synchronous counter settling time = $1 \times t_{pd}$ (just one FF delay)

SHIFT REGISTER TYPES: SISO, SIPO, PISO, PIPO, Universal (bidirectional) Universal SR: $S_i S_o = 00 \rightarrow \text{Hold}$; $01 \rightarrow \text{Shift Right}$; $10 \rightarrow \text{Shift Left}$; $11 \rightarrow \text{Parallel Load}$